Autonomous AI Operator

An agentic AI system that turns a natural-language goal into verified, multi-step actions across connected tools

HackAttack 2026 | AI / ML Track | AMC Engineering College (Autonomous), Dept. of CSE (AIML)
Theme: Agentic AI, MCP and Intelligent Automation

---

1. Overview

People routinely move between browsers, spreadsheets, documents and email to finish a single task. Conventional AI assistants can answer questions or generate text, but the user still has to perform every follow-up action by hand. **Autonomous AI Operator** closes that gap. The user states a goal in plain English; the agent discovers the tools available to it through the Model Context Protocol (MCP), plans the work, executes it, tracks progress, recovers from failures, and verifies the outcome before reporting back.

The guiding principle of the project is: **the LLM plans, but deterministic code owns state, retries, approvals and verification.** The agent never relies on a language model to remember what has been done or to declare success.

2. Problem Statement

Build an autonomous AI agent that understands user goals, plans multi-step tasks and executes them using connected tools or applications. It must track progress, recover from failures and verify results instead of merely suggesting actions. Sensitive operations must require human approval. The prototype demonstrates a practical end-to-end workflow built within the five-hour hackathon window.

3. Key Features

- **Goal to plan:** the LLM converts a natural-language goal into a structured, dependency-aware JSON plan.
- **Dynamic tool discovery:** tools are discovered at runtime through MCP *list_tools*; nothing is hard-coded into the planner.
- **Autonomous execution:** an executor runs each step in order, passing outputs to dependent steps.
- **State persistence:** every task, step and result is stored in SQLite before and after execution, so progress survives crashes and restarts.
- **Failure recovery:** automatic retries with exponential backoff, resume from the point of failure, and no repeated completed work.
- **Idempotent writes:** every write carries an idempotency key and uses upsert semantics, so retries can never create duplicate entries.
- **Result verification:** deterministic code-based checks read results back (for example, the spreadsheet) instead of trusting the LLM.
- **Human-in-the-loop safety:** sensitive actions pause for explicit Approve / Reject; emails are drafts by default.

- **Full audit trail:** every tool call is logged with timestamp, tool, arguments, result and approver.
- **Honest reporting:** the final report lists exactly what succeeded, what was retried and what remains unresolved.

4. Example Scenarios

Scenario A: Internship Research and Application Preparation

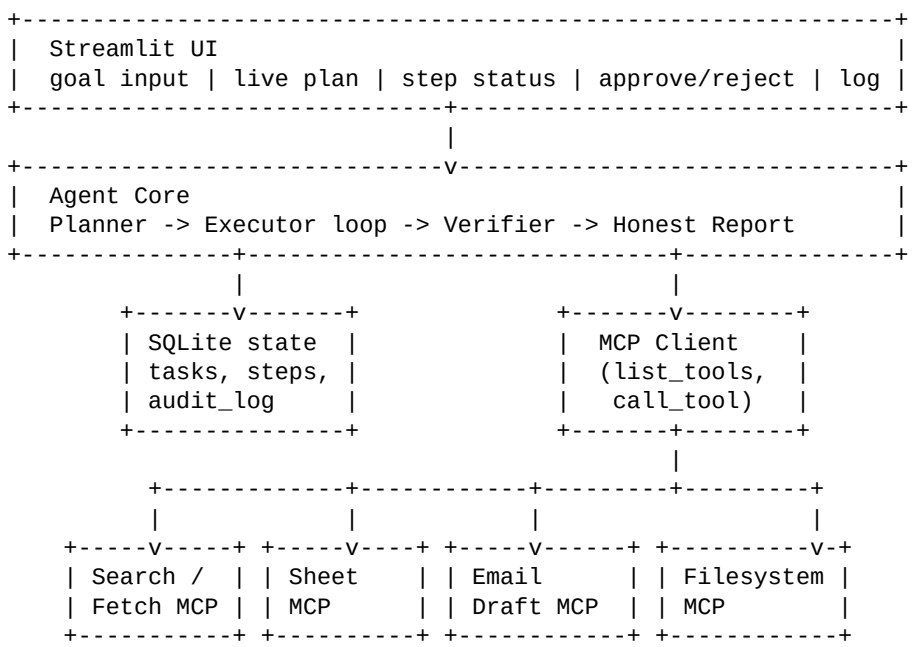
Goal: "Find three internship opportunities suitable for a third-year computer science student, compare their eligibility criteria, save the best matches to a spreadsheet and prepare personalized draft emails for each opportunity."

The agent searches for internships, fetches and extracts details, compares eligibility and deadlines, ranks matches with an explainable score, saves findings with links to a spreadsheet, and creates personalized email drafts. **Outcome:** a verified spreadsheet and email drafts ready for review.

Scenario B: Recovering from a Failed Workflow

After the agent has collected the opportunities, the spreadsheet tool becomes temporarily unavailable. The agent detects the failure, preserves its progress, retries the update without repeating completed actions or creating duplicate rows, verifies the final spreadsheet and reports any unresolved issue instead of falsely claiming success. A *Simulate spreadsheet outage* toggle in the interface lets this scenario be demonstrated live.

5. System Architecture



Component responsibilities

Component	Responsibility
Planner	Receives the goal, the user profile and the discovered tool schemas; returns a JSON plan with step ids, tools, arguments and dependencies.
Executor	Runs steps in dependency order, persists status, retries failures with backoff, skips steps already marked done, and pauses at sensitive steps.

Component	Responsibility
Verifier	Runs deterministic checks on the real outputs (row counts, duplicate keys, required fields, draft existence) and compares them with the plan's success criteria.
MCP Client	Connects to MCP servers, discovers tools at startup and routes tool calls.
State Store	SQLite database holding tasks, steps, results, approvals and the audit log.
MCP Servers	Search/fetch, spreadsheet, email-draft and filesystem tools exposed over MCP.

6. Technology Stack

Layer	Technology
Language	Python 3.10+
LLM	Claude API with tool use (alternatives: Gemini, Groq, or a local model through Ollama)
MCP	Official Python *mcp* SDK (FastMCP) for custom servers; an off-the-shelf fetch/filesystem server for reuse
Spreadsheet	openpyxl, producing a local .xlsx file
Email	Local .eml drafts (sending is a separate, approval-gated action)
State storage	SQLite
Web search	Tavily / Brave / SerpAPI plus a fetch tool
Interface	Streamlit

7. How It Works

Execution flow

1 The user enters a goal in the interface.

2 The agent calls MCP *list_tools* and injects the discovered tool schemas into the planner prompt.

3 The planner returns a structured plan; it is saved to SQLite and displayed live.

4 The executor runs each step, moving it through *pending, running, done* or *failed*.

5 Sensitive steps stop at *awaiting_approval* until the user approves or rejects them.

6 On a tool failure the executor retries with exponential backoff; if retries are exhausted the step is marked failed and all other progress is preserved.

7 The verifier reads back the spreadsheet and drafts and runs deterministic checks.

8 The agent produces an honest final report of successes, retries and unresolved issues.

Example plan

```
[
  {"id": "s1", "tool": "web_search",         "depends_on": []},
  {"id": "s2", "tool": "fetch_page",         "depends_on": ["s1"]},
  {"id": "s3", "tool": "rank_matches",       "depends_on": ["s2"]},
  {"id": "s4", "tool": "sheet_upsert",       "depends_on": ["s3"]},
  {"id": "s5", "tool": "email_create_draft", "depends_on": ["s4"]}
]
```

8. Reliability, Safety and Verification

Error recovery and state persistence

- Step status and results are written to SQLite before and after execution.
- Writes use an idempotency key, for example *hash(company + role + link)*, with upsert behavior in the spreadsheet tool.
- Failed calls are retried three times with exponential backoff.
- After a crash or restart the executor reloads the task and skips every step already marked *done*.

Verification

- The spreadsheet is read back; the row count must match the target and no duplicate keys may exist.
- Every row must contain a link, a deadline and eligibility criteria; missing fields are recorded as "Not found" rather than guessed.
- Each email draft must exist and reference the correct company and role.

Safety and auditability

- Tools are tagged *sensitive* (for example, sending email or overwriting files) and require explicit human approval.
- Emails are created as drafts by default; sending is a separate gated action.
- Every tool call is recorded in an *audit_log* table with timestamp, tool, arguments, result and approver, and is visible in the UI.

9. Project Structure

```
autonomous-ai-operator/
|-- README.pdf
|-- requirements.txt
|-- .env.example
|-- user_profile.json
|-- app.py                  # Streamlit interface
|-- agent/
|   |-- planner.py          # goal -> structured plan
|   |-- executor.py         # step runner, retries, approvals
|   |-- verifier.py         # deterministic result checks
|   |-- mcp_client.py       # tool discovery and calls
|   |-- state.py            # SQLite persistence and audit log
|   `-- report.py           # honest final report
|-- mcp_servers/
|   |-- sheet_server.py     # append_row, find_row, read_all, upsert
|   |-- email_server.py     # create_draft, send_email (gated)
|   `-- search_server.py    # web search and page fetch
|-- data/
|   |-- fallback_internships.json
|   `-- (generated .xlsx, .eml and SQLite files)
`-- docs/                   # architecture slide, demo script
```

10. Setup and Installation

Prerequisites: Python 3.10 or later, an API key for the chosen LLM provider, and (optionally) a search API key.

```
# 1. Clone the repository
git clone <your-repository-url>
cd autonomous-ai-operator

# 2. Create and activate a virtual environment
python -m venv venv
source venv/bin/activate        # Windows: venv\Scripts\activate

# 3. Install dependencies
pip install -r requirements.txt

# 4. Configure environment variables
cp .env.example .env            # then add your API keys

# 5. Run the application
streamlit run app.py
```

Environment variables

Variable	Purpose
LLM_API_KEY	API key for the language model provider
SEARCH_API_KEY	API key for the web search provider (optional in demo mode)
DEMO_MODE	When true, uses the cached fallback internship dataset instead of live search

11. Usage

1 Open the Streamlit app and review or edit the user profile (year, skills, CGPA, location).

2 Enter a goal such as: *Find three internships for a third-year CS student, compare eligibility, save to a spreadsheet and draft emails.*

3 Watch the plan, discovered tools and step statuses update live.

4 Optionally enable *Simulate spreadsheet outage* to demonstrate failure recovery.

5 Approve or reject sensitive steps when prompted.

6 Review the verified spreadsheet, the email drafts, the audit log and the final report.

12. Mapping to Judging Criteria

Criterion	How this project addresses it
Innovation and problem relevance	End-to-end agent that acts across applications for a realistic student workflow.
Planning and autonomous execution	Structured, dependency-aware plans executed step by step without manual intervention.
MCP integration and tool usage	Runtime tool discovery with custom and off-the-shelf MCP servers.
Reliability, recovery, persistence	SQLite state, retries with backoff, idempotent upserts, resume after restart.

Criterion	How this project addresses it
Task completion and verification	Code-based read-back checks and an honest final report.
Safety, oversight, auditability	Approval gates for sensitive tools, draft-only emails, complete audit log.
User experience and demo quality	Live plan view, outage toggle, approval buttons and cached fallback data.

13. Limitations and Future Work

- Live scraping can be unreliable; a cached fallback dataset is provided for demonstrations.
- The prototype uses local .xlsx and .eml files; Google Sheets and Gmail integrations are planned extensions.
- A re-planning step that widens the search when fewer than the target number of valid results is found.
- A graphical step-timeline view and JSON export of the audit log.
- An explainable match-score rubric covering skills, year, deadline and stipend.

14. Team

Name	Role	Contact
<Member 1>	<e.g., Agent core and planner>	<email>
<Member 2>	<e.g., MCP servers>	<email>
<Member 3>	<e.g., UI and demo>	<email>
<Member 4>	<e.g., Verification and docs>	<email>

15. License

This project was developed for the HackAttack hackathon at AMC Engineering College (Autonomous). Add your preferred license (for example, MIT) before making the repository public.